Day 71 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 11, Day 2 reference

Watch: Attention for Neural Networks, Clearly Explained!!! (StatQuest) | Lecture 10: Neural Machine Translation and Models with Attention (Stanford)

`day71_additive_attention.py`, segment by segment, in the order `main()` uses things. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula; the green **SYSTEM DIAGRAM** and the graph are the ones the script saved. Day 69's LSTM code is reused verbatim and not repeated. A glossary is at the end.

MATH BOX LEGEND

Symbols: h_i = encoder state i (the memory) · s = decoder state (the query) · e_i = score · α_i = weight · A = attention size · [a ; b] = a and b side by side

Segment 0 — Parts

The reused read (Day 70 exercise)

MATH

$$\alpha = softmax(e) \text{ over the real slots;} \quad context = \Sigma_i \, \alpha_i \, h_i$$

CODE

```
def masked_softmax(scores, mask):
    s = np.where(mask, scores, -np.inf)
    s = s - s.max(axis=-1, keepdims=True)
    e = np.exp(s)
    return e / e.sum(axis=-1, keepdims=True)
def soft_read(memory, scores, mask):
    w = masked_softmax(scores, mask)
    return np.einsum("mn,mnd->md", w, memory), w
```

- `np.where(mask, scores, -np.inf)` — padding gets minus infinity, so exp() makes its weight exactly 0.
- `np.einsum("mn,mnd->md", w, memory)` — for each sequence m: sum over slots n of weight times slot vector.
- `np.ones(e.shape, bool)` — today every batch has one length, so the mask is all True.

Segment 1 — New building blocks

lstm_step_backward: one step of Day 65's loop

CODE

```
def lstm_step_backward(dh, dc, x, h_prev, c_prev, c, gates, W):
    f, i, g, o = gates
    D = x.shape[1]
    tanh_c = np.tanh(c)
    do = dh * tanh_c
    dc = dc + dh * o * (1 - tanh_c ** 2)
    da = np.concatenate([dc * c_prev * f * (1 - f),
                         dc * g * i * (1 - i),
                         dc * i * (1 - g ** 2),
                         do * o * (1 - o)], axis=1)
    z = np.concatenate([x, h_prev], axis=1)
    dz = da @ W.T
    return dz[:, :D], dz[:, D:], dc * f, z.T @ da, da.sum(axis=0)
```

- `dc = dc + dh * o * (1 - tanh_c ** 2)` — the cell state gets gradient from the next step and through h = o tanh(c).
- `dz[:, :D], dz[:, D:]` — split the gradient of [x ; h_prev] back into its two parts.
- `why a step version?` — the decoder's input at each step depends on its own previous state through attention, so Day 64's loop cannot run first and be differentiated afterwards in one call.

additive_scores and its backward pass

MATH

$$e_i = v_a \cdot \tanh(s\,W_a + k_i), \quad k_i = h_i\,U_a + b_a$$
$$da_i = de_i\,v_a\,(1 - t_i^2); \quad ds = (\Sigma_i\, da_i)\,W_a^T$$

CODE

```
def additive_scores(s_prev, keys, Wa, va):
    t = np.tanh((s_prev @ Wa)[:, None, :] + keys)
    return t @ va, t
def additive_scores_backward(de, t, s_prev, Wa, va):
    dva = np.einsum("mn,mna->a", de, t)
    da = de[:, :, None] * va * (1 - t ** 2)
    dq = da.sum(axis=1)
    return dq @ Wa.T, da, s_prev.T @ dq, dva
```

- `(s_prev @ Wa)[:, None, :] + keys` — broadcasting: one query per sequence is added to each of its n keys.
- `t @ va` — a vector product over the last axis: one score per encoder step.
- `de[:, :, None] * va * (1 - t ** 2)` — back through e = t . va, then through tanh.
- `da.sum(axis=1)` — the query was added at every encoder step, so its gradient sums over them.

Segment 2 — The model

AttnSeq2Seq: weights, encode and one step

CODE

```
class AttnSeq2Seq:
    def __init__(self, hidden=32, attn=32, seed=0, forget_bias=1.0, lr=0.5, clip=1.0,
        init_from_encoder=True):
        rng = np.random.RandomState(seed)
        H, A = hidden, attn
        self.H, self.lr, self.clip, self.init_from_encoder = H, lr, clip, init_from_encoder
        self.P = {"We": rng.randn(V + H, 4 * H) / np.sqrt(V + H), "be": np.zeros(4 * H),
                  "Wd": rng.randn(V + 2 * H, 4 * H) / np.sqrt(V + 2 * H), "bd": np.zeros(4 * H),
                  "Wa": rng.randn(H, A) / np.sqrt(H), "Ua": rng.randn(H, A) / np.sqrt(H), "ba":
                      np.zeros(A),
                  "va": rng.randn(A) / np.sqrt(A),
                  "Why": rng.randn(2 * H, V) / np.sqrt(2 * H), "by": np.zeros(V)}
        self.P["be"][:H] = forget_bias; self.P["bd"][:H] = forget_bias
    def encode(self, Xs):
        m = Xs.shape[0]
        z = np.zeros((m, self.H))
        Hs, Cs, gates = lstm_forward(Xs, z, z, self.P["We"], self.P["be"])
        keys = Hs @ self.P["Ua"] + self.P["ba"]
        s, c = (Hs[:, -1], Cs[:, -1]) if self.init_from_encoder else (z, z)
        return Hs, Cs, gates, keys, s, c
    def step(self, y_onehot, s, c, Hs, keys):
        e, t = additive_scores(s, keys, self.P["Wa"], self.P["va"])
        ctx, alpha = soft_read(Hs, e, np.ones(e.shape, bool))
        x = np.concatenate([y_onehot, ctx], axis=1)
        s_new, c_new, gates = lstm_step_forward(x, s, c, self.P["Wd"], self.P["bd"])
        out = np.concatenate([s_new, ctx], axis=1)
        return s_new, c_new, dict(s_prev=s, c_prev=c, t=t, alpha=alpha, ctx=ctx, x=x, s=s_new,
            c=c_new, gates=gates, out=out)
```

- `"Wd": rng.randn(V + 2 * H, ...)` — the decoder's input is the previous token AND the context: V + H numbers, plus H for h.
- `keys = Hs @ self.P["Ua"] + self.P["ba"]` — each encoder state's key is computed once per sequence, not once per decoder step.
- `... if self.init_from_encoder else (z, z)` — the decoder may start from the encoder's end (Day 69) or from zeros.
- `np.concatenate([y_onehot, ctx], axis=1)` — [y ; context]: what the decoder LSTM reads at this step.
- `np.concatenate([s_new, ctx], axis=1)` — the output layer sees the new state and the same context.

loss_and_grads: the backward loop through attention

MATH

$$d\,h_i \mathrel{+}= \alpha_i\, d\,c; \quad d\alpha_i = d\,c \cdot h_i; \quad de = \alpha \times (d\alpha - \Sigma_i\, \alpha_i\, d\alpha_i)$$

CODE

```
for j in reversed(range(T)):
    st = steps[j]
    ds = ds + douts[j][:, :H]
    dctx = douts[j][:, H:].copy()
    dx, ds_prev, dc_prev, dW, db = lstm_step_backward(ds, dc, st["x"], st["s_prev"], st["c_prev"],
        st["c"], st["gates"], P["Wd"])
    g["Wd"] += dW; g["bd"] += db
    dctx += dx[:, V:]
    alpha = st["alpha"]
    dHs += alpha[:, :, None] * dctx[:, None, :]
    dalpha = np.einsum("mh,mnh->mn", dctx, Hs)
    de = alpha * (dalpha - (alpha * dalpha).sum(axis=1, keepdims=True))
    dsp, dk, dWa, dva = additive_scores_backward(de, st["t"], st["s_prev"], P["Wa"], P["va"])
    g["Wa"] += dWa; g["va"] += dva; dkeys += dk
    ds, dc = ds_prev + dsp, dc_prev
```

- `dctx += dx[:, V:]` — the context reached the loss twice: through the output layer and through the decoder's input.
- `dHs += alpha[:, :, None] * dctx[:, None, :]` — context = $\Sigma\, \alpha_i\, h_i$, so every encoder state gets $\alpha_i\, d$ context.
- `alpha * (dalpha - (alpha * dalpha).sum(axis=1, keepdims=True))` — the softmax's backward pass (Day 70 exercise).
- `ds_prev + dsp` — the previous state fed the LSTM AND the scores, so both gradients add.

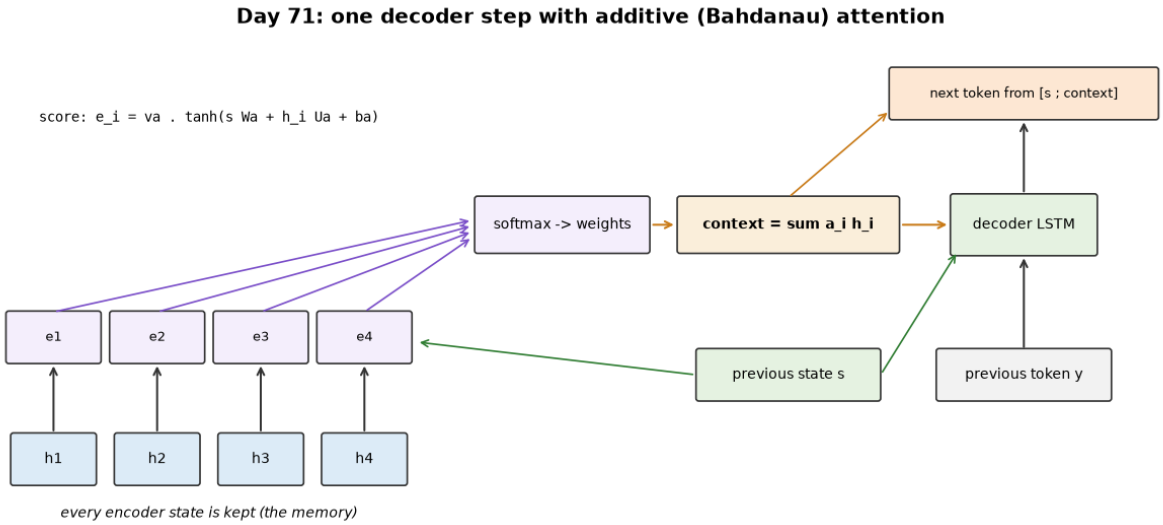
After the loop: the keys and the encoder

CODE

```
g["Ua"] = np.einsum("mnh,mna->ha", Hs, dkeys)
g["ba"] = dkeys.sum(axis=(0, 1))
dHs += dkeys @ P["Ua"].T
z = np.zeros((m, H))
dh_last, dc_last = (ds, dc) if self.init_from_encoder else (z, z)
g["We"], g["be"], _, _ = lstm_backward(dHs, dh_last, dc_last, Xs, z, z, P["We"], Hs, Cs, gates)
```

- `np.einsum("mnh,mna->ha", Hs, dkeys)` — keys = Hs Ua: sum over sequences and steps of h outer d key.
- `dHs += dkeys @ P["Ua"].T` — the encoder states made the keys too, so they get a third gradient.
- `lstm_backward(dHs, dh_last, dc_last, ...)` — Day 69's backward pass, now with a gradient at EVERY encoder step.

SYSTEM DIAGRAM



One decoder step: scores for every encoder state, weights, the context, and the decoder LSTM.

generate: greedy decoding, keeping the weights

CODE

```
def generate(self, Xs, n_steps, return_weights=False):
    Hs, _, _, keys, s, c = self.encode(Xs)
    token = np.full(len(Xs), GO)
    out, weights = [], []
    for _ in range(n_steps):
        s, c, st = self.step(np.eye(V)[token], s, c, Hs, keys)
        token = (st["out"] @ self.P["Why"] + self.P["by"]).argmax(axis=1)
        out.append(token); weights.append(st["alpha"])
    pred = np.stack(out, axis=1)
    return (pred, np.stack(weights, axis=1)) if return_weights else pred
```

- `weights.append(st["alpha"])` — one row of attention weights per output step, for Day 72.
- `np.stack(weights, axis=1)` — (sequences, output steps, input steps).
- `(a, b) if cond else a` — return two things only when asked.

Segment 3 — Experiments

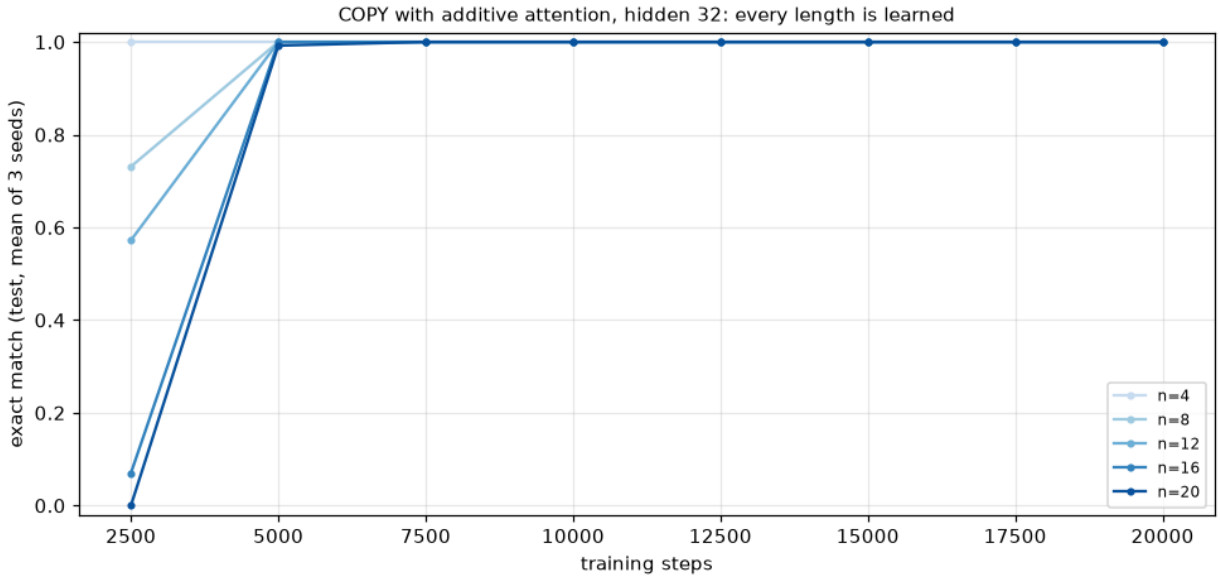
Timing one step of each model

CODE

```
t_plain = best(lambda: plain.loss_and_grads(Xs, Xd, Y))
t_attn = best(lambda: attn.loss_and_grads(Xs, Xd, Y))
```

- `lambda: plain.loss_and_grads(Xs, Xd, Y)` — a zero-argument function so best() can call it repeatedly (Day 66).
- `the ratio` — both timed in the same run, under the same conditions; absolute times vary between runs.

REAL GRAPH



Real graph from the script: exact match during training, every length.

Quick Syntax Glossary — new or reinforced constructs

- `x[:, None, :] + K` — insert an axis so one row broadcasts against many.
- `np.einsum("mn,mnd->md", w, M)` — a weighted sum written as an index formula.
- `np.where(mask, a, -np.inf)` — replace masked entries with minus infinity.
- `dict(a=1, b=2)` — a dictionary built with keyword arguments.
- `for j in reversed(range(T))` — walk the steps backwards.
- `x.sum(axis=(0, 1))` — sum over two axes at once.
- `lambda: f(a)` — a function with no arguments that calls f(a).

End of Day 71 syntax reference. For the experiments and what they showed, see the Day 71 complete guide.